

Inference Engineering Assignment

Part A: Theory

1. Latency vs. Throughput

Latency is the time taken to produce a response for a *single* request (from the moment the request arrives to the moment the output is returned). It is usually measured in milliseconds and matters most for the end-user experience — e.g. how quickly a chatbot responds to one message.

Throughput is the number of requests (or tokens) a system can process per unit time *across all concurrent requests*, usually measured in requests/second or tokens/second. It matters most for overall system capacity and cost efficiency.

These two are frequently in tension. Techniques such as batching multiple requests together increase throughput (the GPU/CPU does more useful work per pass over the weights), but they increase the latency of any individual request, since that request may have to wait for other requests in the batch and the batch as a whole takes longer to process than a single sample would alone.

Which matters more? It depends entirely on the product:

- Interactive applications (chat assistants, code completion, voice assistants) are latency-sensitive — users notice delays of even a few hundred milliseconds.
- Offline or bulk-processing workloads (batch summarization of a document corpus, embedding generation for a search index) are throughput-sensitive — total wall-clock time to finish the whole job matters more than any single item’s turnaround.

In practice, serving systems expose a latency/throughput trade-off knob (batch size, max batch wait time) and tune it to the target use case. Metrics like *time-to-first-token* (TTFT) and *time-per-output-token* (TPOT) are used together with throughput to fully characterize performance, since raw throughput alone can hide poor per-request latency.

2. KV Cache

Autoregressive transformer models generate text one token at a time, and each new token’s prediction depends on attending to all previous tokens. Naively, this means at every generation step the model would need to recompute the Key (K) and Value (V) projections for *every* token in the sequence so far, even though most of those tokens haven’t changed since the last step. This makes generation cost grow quadratically with sequence length.

The **KV cache** avoids this redundant work: at each layer, once the K and V vectors for a token are computed, they are stored (cached) in memory. On the next generation step, the model only needs to compute Q, K, V for the *newest* token, and then attends to it plus the cached K/V vectors of all prior tokens — it does not recompute K/V for tokens it has already processed.

This turns each generation step from an $O(n)$ recomputation (over the growing sequence length n) into an $O(1)$ per-token compute cost (plus the cost of attending over the cache), so generation becomes roughly linear in total sequence length rather than quadratic. The trade-off is memory: the KV cache grows linearly with sequence length, batch size, number of layers, and number of attention heads, and for long contexts or large batches it can become the dominant memory consumer during serving (this is exactly the problem optimizations like vLLM’s PagedAttention, described in Q4, target).

3. Quantization

Quantization is the process of representing a model’s weights (and sometimes activations) using fewer bits than the original floating point format — commonly converting 32-bit floats (FP32) or 16-bit floats (FP16) down to 8-bit integers (INT8) or 4-bit integers (INT4). This reduces model size, memory bandwidth requirements, and often inference latency, since integer arithmetic is cheaper and moving fewer bytes through memory is faster.

Trade-off: accuracy vs. compression

- **INT8** typically causes a very small accuracy drop (often under 1% on standard benchmarks) while giving roughly a 4x reduction in model size compared to FP32, and noticeably faster CPU inference.
- **INT4** is far more aggressive (8x reduction vs. FP32) but the smaller numeric range increases quantization error, and accuracy degradation becomes more noticeable, especially without careful calibration or techniques like GPTQ/AWQ that protect salient weights. INT4 is more likely to hurt performance on tasks requiring fine-grained numeric precision or long reasoning chains.

The general pattern: fewer bits $\rightarrow$ smaller and faster, but a higher risk of accuracy loss, and the exact accuracy hit depends heavily on the quantization method (dynamic vs. static, per-tensor vs. per-channel, with or without calibration data).

Formula

Standard *affine (uniform) quantization* maps a floating point value x to an integer q using a scale s and zero-point z :

$$q = \text{round}\left(\frac{x}{s}\right) + z, \quad s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$$

and dequantization (recovering an approximate float) is:

$$\hat{x} = s \cdot (q - z)$$

Worked example

Suppose a weight tensor has values ranging from $x_{\min} = -2.0$ to $x_{\max} = 2.0$, and we quantize to **unsigned INT8**, so $q_{\min} = 0$, $q_{\max} = 255$.

Step 1 — compute scale:

$$s = \frac{2.0 - (-2.0)}{255 - 0} = \frac{4.0}{255} \approx 0.01569$$

Step 2 — compute zero-point:

$$z = \text{round} \left(q_{\min} - \frac{x_{\min}}{s} \right) = \text{round} \left(0 - \frac{-2.0}{0.01569} \right) = \text{round}(127.5) = 128$$

Step 3 — quantize a sample value, e.g. $x = 0.75$:

$$q = \text{round} \left(\frac{0.75}{0.01569} \right) + 128 = \text{round}(47.8) + 128 = 48 + 128 = 176$$

Step 4 — dequantize back to float to see the error introduced:

$$\hat{x} = 0.01569 \times (176 - 128) = 0.01569 \times 48 \approx 0.7531$$

So the original value 0.75 becomes 0.7531 after the round-trip — a quantization error of about 0.003, which is representative of the small, bounded error INT8 quantization introduces per weight. Errors like this accumulate across millions of weights and layers, which is why aggressive quantization (INT4 and below) needs more careful calibration to keep the compounded error from noticeably degrading model outputs.

4. Inference Serving Framework Example

vLLM is a widely used open-source LLM inference and serving engine. One of its key optimizations is **PagedAttention**: instead of allocating one large, contiguous block of GPU memory for each sequence’s KV cache (which wastes memory to fragmentation and to over-provisioning for a worst-case sequence length), PagedAttention manages the KV cache in small, fixed-size, non-contiguous “pages,” analogous to how an operating system manages virtual memory with paging. This lets vLLM pack far more concurrent sequences into the same GPU memory, dramatically increasing achievable throughput (batch size) for a given hardware budget, with minimal memory waste and support for efficient sharing of KV cache pages across sequences (e.g. for parallel sampling or shared prompt prefixes).